Transformer Architecture Working

Vocab = ["I", "love", "cats", "dogs", "and", "."] = 6 Words

Dimensions ⇒ 4 , No of attention head = 1

Input to the model = [I, love]

Step 1 :- Tokenization.

It is a process where we convert a large sentence into small chunk called Tokens.

It performs 2 operations :-

i) Convert large sentence into Tokens

ii) Assign them a unique no called Token ID

Sentence = [I love]

After Tokenization = [0, 1]

Vocab

TokenId	Token
0	I
1	love
2	cats
3	dogs
4	and
5	.

Step 2 :- Token Embedding

At the training time, for each token in the vocab, we create an embedding which we store in a Table called lookup Table

Based on the Token Id, it finds the static embedding of the token which model learns at the time of training.

Sentence Tokens ⇒ [0, 1]

Token(0)(I) = [1.0, 00, 0.5, -0.5]

Token(1)(love) = [0.5, 1.0, -0.5, 0.5]

lookup Table

Token ID	Embedding
0	[1.0, 00, 0.5, -0.5]
1	[0.5, 1.0, -0.5, 0.5]
2	[0.3, 0.8, 0.2, 0.2]
3	[0.4] 0.7, 0.3, 0.0
4	[0.1, 0.2, 0.0, 0.9]
5	[0.0, -0.3, 0.6, 0.4]

Now, Input Matrix,

$$X = \begin{bmatrix} 1.0 & 0.0 & 0.5 & -0.5 \\ 0.5 & 1.0 & -0.5 & 0.5 \end{bmatrix} \begin{matrix} \Leftarrow I \\ \Leftarrow \text{love} \end{matrix}$$

Shape = 2X4

Step 3 ⇒ Add Position embedding

It is a vector that represents the position of a word in the sentence.

$$X = \begin{bmatrix} (1.0, 0.0, 0.5, 0.5) + (0.1, 0.1, 0.0, 0.0) \\ (0.5, 1.0, -0.5, 0.5) + (0.0, 0.0, 0.1, 0.1) \end{bmatrix}$$

$$X = \begin{bmatrix} 1.2 & 0.1 & 0.5 & -0.5 \\ 0.5 & 1.0 & -0.4 & 0.6 \end{bmatrix}$$

At the time of Training

Position	embedding
0	[0.1, 0.1, 00, 0.0]
1	[0.0, 0.0, 0.1, 0.1]

Step 4 ⇒ Calculate Q, k, V. (Self attention)

Self attention is a mechanism by which we convert the static embedding into the contextual embedding for each word in the sentence.

It performs two thing:-

i] Calculate attention Scores

ii] Combines information from various positions.

For each words, we Calcute three matrics. Q, k, V

Query (Q) ⇒ It represents the vector for which we are calculating the attention.

key (k) ⇒ It represents all the other words present in the sentence to Calcute the attention Score.

Value (V) ⇒ It stores the actual value.

ARCTIC

USA

ne of training, we get three learned matrices called W-Q, W-k, W-V which ① we to Calculate Q, k, V.

$$Q=\begin{bmatrix}0.2 & 0.1 & -0.3 & 0.4\\ -0.1 & 0.3 & 0.2 & -0.1\\ 0.3 & -0.2 & 0.1 & 0.3\\ -0.4 & 0.1 & 0.3 & -0.2\end{bmatrix} \quad W\text{-}k=\begin{bmatrix}0.1 & -0.2 & 0.3 & 0.1\\ 0.3 & 0.1 & -0.1 & 0.2\\ -0.2 & 0.3 & 0.2 & -0.1\\ 0.1 & -0.3 & 0.1 & 0.3\end{bmatrix} \quad W\text{-}V=\begin{bmatrix}0.3 & 0.1 & -0.1 & 0.2\\ -0.1 & 0.2 & 0.3 & -0.1\\ 0.2 & -0.1 & 0.1 & 0.3\\ -0.2 & 0.3 & -0.2 & 0.1\end{bmatrix}$$

$$Q=\begin{bmatrix}1.1 & 0.1 & 0.5 & -0.5\\ 0.5 & 1.0 & -0.4 & 0.6\end{bmatrix}\begin{bmatrix}0.2 & 0.1 & -0.3 & 0.4\\ -0.1 & 0.3 & 0.2 & -0.1\\ 0.3 & -0.2 & 0.1 & 0.3\\ -0.4 & 0.1 & 0.3 & -0.2\end{bmatrix}=\begin{bmatrix}0.56 & -0.23 & 0.21 & -0.18\\ 0.36 & 0.39 & 0.09 & -0.34\end{bmatrix}$$

$$K=\begin{bmatrix}1.1 & 0.2 & 0.05 & -0.5\\ 0.5 & 1.0 & -0.4 & 0.6\end{bmatrix}\begin{bmatrix}0.1 & -0.2 & 0.3 & 0.1\\ 0.3 & 0.1 & -0.01 & 0.2\\ -0.2 & 0.3 & 0.2 & -0.1\\ 0.1 & -0.3 & 0.1 & 0.3\end{bmatrix}\Rightarrow\begin{bmatrix}-0.01 & 0.09 & 0.37 & -0.07\\ 0.49 & -0.30 & 0.03 & 0.47\end{bmatrix}$$

$$V=\begin{bmatrix}1.1 & 0.1 & 0.05 & -0.5\\ 0.5 & 1.0 & -0.4 & 0.6\end{bmatrix}\begin{bmatrix}0.3 & 0.1 & -0.1 & 0.2\\ -0.1 & 0.2 & 0.3 & -0.1\\ 0.2 & -0.1 & 0.1 & 0.3\\ -0.2 & 0.3 & -0.2 & 0.1\end{bmatrix}=\begin{bmatrix}0.52 & -0.07 & 0.07 & 0.31\\ 0.13 & 0.47 & 0.09 & 0.06\end{bmatrix}$$

Step 5 ⇒ Compute Attention Score.

Attention Score is a number which tell us the relevance of a word from another word in a Sentence

$$\text{Attention Score} = (Q \times K^T)$$

Where Q = Query and K^T = key Transpose

Q

$$\text{Attention Score} = \begin{bmatrix} 0.56 & -0.23 & 0.21 & -0.18 \\ -0.36 & 0.39 & 0.09 & -0.31 \end{bmatrix} \begin{bmatrix} -0.01 & 0.49 \\ 0.09 & -0.30 \\ 0.37 & 0.03 \\ 0.07 & 0.47 \end{bmatrix} K^T$$

$$= \begin{bmatrix} 0.0640 & 0.2651 \\ 0.0958 & -0.4505 \end{bmatrix} \begin{matrix} \leftarrow \text{"I" attends to} \\ \leftarrow \text{"love" attends to} \end{matrix}$$

"I" "love"

Step 6 ⇒ Scaling

Scaling helps use to prevent attention score growing too large

Here, We take the attention score and divide by $\sqrt{d_k}$

Problems that Scaling Solves ÷

i] Gradient exploding

ii] Softmax Saturation ⇒ most of the attention weight assigned to a single token.

⇒

$\sqrt{d_k} = \sqrt{4} \Rightarrow 2$

$$\text{Attention Score} = \begin{bmatrix} \frac{0.0640}{2} & \frac{0.2651}{2} \\ \frac{0.0958}{2} & \frac{-0.4505}{2} \end{bmatrix} \Rightarrow \begin{bmatrix} 0.0320 & 0.1326 \\ 0.0479 & -0.2253 \end{bmatrix}$$

Step 7 ⇒ Apply Casual mask

It helps to prevent model to look at the future tokens. I love

After Mask ⇒ $\begin{bmatrix} 0.0320 & -\infty \\ 0.0479 & -0.2253 \end{bmatrix}$ ⇒ I can see only I

I can see both I and see

apply softmax attention score into probabilities that sum to 1.

$$\text{softmax}(x-i) = \frac{\exp(x-i)}{\sum \exp(x-i)}$$

$$= \begin{bmatrix} 0.0320 & -\infty \\ 0.0479 & -0.2251 \end{bmatrix}$$

$\Rightarrow \exp(0.0320) = 1.0325 \qquad \exp(-\infty) = 0.000$

$\Rightarrow$ Sum $= 1.0325 + 0.000 = 1.0325$

$\Rightarrow \exp(0.0479) = 1.0491 \qquad \exp(-0.2251) = 0.7983$

$\Rightarrow$ Sum $= 1.0491 + 0.7983 = 1.8474$

$$\Rightarrow \begin{bmatrix} 1.0325/1.0325 & 0.00/1.0325 \\ 1.0491/1.8474 & 0.7983/1.8474 \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} 1.000 & 0.000 \\ 0.5678 & 0.4322 \end{bmatrix}$$

Step 9 $\Rightarrow$ Compute attention output ($w \times v$)

$$\text{Attention O/P} = \begin{bmatrix} 1.000 & 0.000 \\ 0.5678 & 0.4322 \end{bmatrix} \times \begin{bmatrix} 0.52 & 0.07 & 0.07 & 0.31 \\ -0.15 & 0.47 & 0.09 & -0.6 \end{bmatrix}$$

$$= \begin{bmatrix} 0.052 & -0.0700 & 0.0700 & 0.3100 \\ 0.238 & 0.1634 & 0.078 & 0.1501 \end{bmatrix}$$

I

like

Step 10 :- Output Projection.

The attention o/p that we got is still raw. It is not directly used with other layers.

Role :- Some dimension might overlap in meaning Redundancy.

Proj_Out = Self attent o/p * Wo

Where Wo is the learned weight matrix.

⇒ Used to transform the self attention into a better representation.

⇒ It rearranged the info.

$$= \begin{bmatrix} 0.5800 & -0.0700 & 0.0700 & 0.3100 \\ 0.2350 & 0.1634 & 0.3786 & 0.1501 \end{bmatrix} \times \begin{bmatrix} 0.2 & -0.1 & 0.3 & -0.4 \\ 0.1 & 0.3 & -0.2 & 0.1 \\ -0.3 & 0.2 & 0.1 & 0.3 \\ 0.4 & -0.1 & 0.3 & -0.2 \end{bmatrix}$$

$$= \begin{bmatrix} 0.200 & -0.0900 & 0.2700 & -0.2560 \\ 0.0988 & 0.0266 & 0.0894 & -0.0823 \end{bmatrix} \in \text{love}$$

Steps 11 ⇒ Add Residual Connection

Here, We add our original Static embedding Vector to the output of Self attention

Need :-

i) It might be possible while calculating the o/p Projet, we might miss some of the actual info while balancing the weights.

ii) Resolves Vanishing Gradient :-

Gradient is a no which tells how to reduce weights to reduce error.

It's a problem when Gradient becomes too Small

$$\begin{bmatrix} 200 & -0.0900 & 0.2700 & -0.2560 \\ 0.0988 & 0.0266 & 0.0894 & -0.0425 \end{bmatrix} + \begin{bmatrix} 1.0 & 0.0 & 0.5 & -0.5 \\ 0.5 & 1.0 & -0.5 & 0.5 \end{bmatrix}$$

$$= \begin{bmatrix} 1.300 & -0.0900 & 0.7700 & -0.7560 \\ 0.5988 & 1.0266 & -0.3106 & 0.5177 \end{bmatrix} \Rightarrow I \quad (7)$$
$$\Rightarrow love$$

Step 12 ⇒ layer Normalization.

Normalization is a process in which we adjust no so that they can be centered around 0 and scale to consistent range.

It adjust the weights of the vector into same scale

Problem without normalization :-

ex ⇒ $V_1 = [1, 2, 3]$, $V_2 = [10, 20, 30]$

⇒ Same pattern but different scale.

⇒ model can treat the second vector more Imp as it has high weight

layer Normalization $= \frac{x_i - \mu}{\sqrt{\sigma^2}}$ where μ = mean, σ = Standar deviation

μ = It tells the center.

σ = how far a number is from center.

mean $\mu_1 = (1.300 - 0.0900 + 0.7700 - 0.7560)/4 = 0.3060$

$\mu_2 = (0.5988 + 1.0266 - 0.3106 + 0.5177)/4 = 0.1481$

$\sigma^2 = (x_i - \mu)^2 / N$

Deviation $= ((1.300, -0.0900, 0.7700, -0.7560) - 0.3060)$

$(x_i - \mu) = (0.9940 \quad -0.396 \quad 0.464 \quad -1.062)$

Deviation$_2$ $= ((0.5988, 1.0266, -0.3106, 0.5177) - 0.3060$

$= (0.1407, 0.5685, -0.7687, 0.0596)$

$\sigma_1 = (0.9940^2 + 0.3960^2 + 0.4640^2 + 1.0620^2)$

$= 0.6220$

$\sigma_2 = (0.1407^2 + 0.5685^2 + (-0.7687)^2 + (0.0596)^2)$

$= 0.2344$

Normalization for layer 1 'I' $= \frac{(0.9940, 0.3960, 0.4640, 1.0620)}{\sqrt{0.6220}}$

$= [1.2602, -0.5021, 0.5883, -1.3464]$

Normlization for layer 2 $= \frac{(0.1407, 0.5685, 0.7687, 0.0596)}{\sqrt{0.2344}}$

$= [0.2906, 1.1742, -1.5880, 0.1231]$

$$X = \begin{bmatrix} 1.2602, -0.5021, 0.5883, -1.3464 \\ 0.2906, 1.1742, -1.5880, 0.1231 \end{bmatrix}$$

Step 13 - Feed Forward Neural Network

A small Neural Network which process each word Individually.

It add non-linearity which helps model to learn Complex patterns.

~~Next of~~ For. each token coming out of the self attention is applied. On the 2 Neural Network.

This step involves the pattern learning.

Feed Forward NN

mixes the information between words

FFN applies the non-linearity + transformation to each word, allowing model to learn complex patterns and refines the contextual info

Ex ⇒ Real life analogy

Step 1 = Collect ideas from different people (W-FF1)

Step 2 = Reject bad ideas (ReLU)

Step 3 = Combines good idea into final decision (W-FF2)

FFN applies the non-linear transformation to each token's representation, allowing the model to refine and learn complex patterns.

We have 2 learned weight matrics for each NN.

$$W\text{-}FF_1 = \begin{bmatrix} 0.5 & -0.3 & 0.2 & 0.4 \\ -0.2 & 0.4 & 0.1 & 0.3 \\ 0.3 & -0.2 & 0.5 & -0.1 \\ -0.1 & 0.3 & 0.2 & -0.4 \end{bmatrix} \quad W\text{-}FF_2 = \begin{bmatrix} 0.3 & 0.1 & -0.2 & 0.4 \\ -0.1 & 0.2 & 0.3 & -0.1 \\ 0.4 & -0.3 & 0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 & 0.3 \end{bmatrix}$$

layer 1 ⇒ $x \times W\text{-}FF_1$.

$$= \begin{bmatrix} 1.2602 & -0.5021 & 0.5883 & -1.3464 \\ 0.2906 & 1.1742 & -1.588 & 0.1231 \end{bmatrix} \times \begin{bmatrix} 0.5 & -0.3 & 0.2 & 0.4 \\ -0.2 & 0.4 & 0.1 & 0.3 \\ 0.3 & -0.2 & 0.5 & -0.1 \\ -0.1 & 0.3 & 0.2 & -0.4 \end{bmatrix}$$

We are just doing for "I"

hidden layer 1 ⇒ $\begin{bmatrix} 1.0446 & -1.1005 & 0.3271 & 0.8333 \end{bmatrix}$

After Relu ⇒ max(0, x)

$$= \begin{bmatrix} 1.0416 & 0 & 0.3271 & 0.8333 \end{bmatrix}$$

layer 2 ⇒ Relu(hidden) X W_FF2

$= [0.6100, 0.0894, -0.2589, 0.7020]$

⇒ Add Residual & Normalize Again. Why?

⇒ Residual keeps old knowledge
Normalize keeps the weights stable & balanced.

Final ("i") = layer Norm (Normalization o/p of self attention + FFN_out ('I'))

= layerNorm ([1.2602 + 0.6100, -0.5021 + 0.0894, 0.5983 - 0.2589, -1.3464 + 0.7020])

= layerNorm ([1.8702, -0.4127, 0.3394, -0.6144])

= [1.44, -0.80, -0.04, -1.20] (approximate)

The same process will be applied for the token "love"

$$\begin{bmatrix} 1.44 & -0.80 & -0.04 & -1.20 \\ 0.8 & 1.2 & -0.5 & 0.3 \end{bmatrix} \Rightarrow \text{love}$$

Step 14 :- Compute logits (Final Linear Projection)

logits ⇒ Raw score for each words before converting into probabilities

L_M This linear Projection provides a score to each word in the Vocab.

L_M head is responsible to do that.

It store the token embedding in the Vocab

$$W_lm_head = \begin{bmatrix} 0.5 & 0.1 & -0.3 & 0.2 & 0.4 & -0.1 \\ -0.2 & 0.4 & 0.1 & -0.3 & 0.2 & 0.5 \\ 0.3 & -0.1 & 0.5 & 0.4 & -0.2 & 0.1 \\ -0.4 & 0.3 & 0.2 & 0.1 & 0.5 & -0.2 \end{bmatrix} \Rightarrow \text{Embed_d}$$

I love Sentences .

X

Vocab size.

Last — Processed token × W_LM head

$$[0.9 \quad 1.1 \quad -0.5 \quad 0.3] \times \begin{bmatrix} 0.5 & 0.1 & -0.3 & 0.2 & 0.4 & -0.1 \\ -0.2 & 0.4 & 0.1 & -0.3 & 0.2 & 0.5 \\ 0.3 & -0.1 & 0.5 & 0.4 & -0.2 & 0.1 \\ -0.4 & 0.5 & 0.2 & 0.1 & 0.5 & -0.2 \end{bmatrix}$$

$$= [-0.09, 0.66, -0.32, -0.34, 0.79, 0.36]$$

I love cats dogs and .

Step 15:- Soft max (convert logits into probabilities

P(I) = 0.1154 (11.5%) P(dog) = 0.0798 (9%)
P(love) = 0.2442 (24.4%) P(and) = 0.2782 (27.8) highest
P(cats) = 0.916 (9.2%) P(.) = 0.1809 (18.1%)

Probability Distribution Table

Token	Probability bar
I	11.5%
love	24.4%
dogs	9.2%
cats	9.0%
and	27.8%
.	18.1%

Step 16 - Token Selection

Greedy decoding => Mode pick Token with the highest probability

Then process then repeats :-

=> Compute new k,v for "and" only

=> Reuse Kached k,v for "I" and "love" from kv Cache

=> Run attention with three tokens

=> Predict the next token